CISM 621 Practical 4: prompt-driven recommender construct (netflix-like)

Student Name: Mafalo David Mothiba

Student No: 38724839
Date: 26 October 2025

A. Prompt Documentation (Evidence of AI Usage)

1. All prompts used

Prompt1

"IntroductionEach student must design a prompt-driven recommender construct that simulates the behaviour of asimplified Netflix system. The construct should understand a user's text request (e.g., "light romanticcomedies from the 1990s") and recommend suitable movies from a dataset. The recommender logic canbe implemented using Flask and Python scripts, or Jupyter Notebook.Students must use prompt engineering to generate key parts of the system (for example: datasetstructuring, filter logic, explanation generation, or interface text).Every step of AI involvement must be documented with screenshots and written explanations.InstructionsA. Working ConstructDesign and implement a GUI based recommender engine that:• Accepts user input (text query) and returns relevant movie recommendations.• Uses a small dataset (minimum 300 entries with title, genre, cast, plot, and year).• Demonstrates basic logic for filtering or ranking recommendations.• Displays recommendations with short explanations.• It can be run as a simple Flask app using a python notebook or script." i have this assignment , so please explain in simple english what is needed the steps i should follow

Results

Assignment Summary (Simple English)

You need to **create a small Netflix-style movie recommender** that:

1. Understands a user's text query (like "romantic comedies from the 1990s").
2. Suggests movies from a dataset of **at least 300 movies** (with title, genre, cast, plot, and year).
3. Shows recommendations in a **simple GUI** (text box + button).
4. Provides **short explanations** for each recommendation.
5. Uses **AI prompts** to help build the dataset, ranking logic, explanations, or interface.
6. **Documents all AI usage** with screenshots and explanations.

Steps to Complete the Assignment

1. **Create the dataset** – at least 300 movies with all necessary fields.
2. **Build recommendation logic** – match user queries to movies using keywords or TF-IDF similarity.
3. **Generate explanations** – short, readable sentences explaining why each movie was recommended.
4. **Create a GUI** – allow users to type queries and see recommendations.
5. **Use AI prompts** – to generate dataset, explanations, or interface text.
6. **Document AI usage** – include prompts, AI output screenshots, and explanations.
7. **Test your system** – try multiple queries to make sure it works correctly.
8. **Submit** – the working system, dataset, and documentation.

If you want, I can also make a **super clear, one-page checklist version** that you can follow step by step while working on the project. It makes it almost impossible to miss anything.

Do you want me to make that checklist?

Prompt2

so for dataset should i create mine or download it from online?, which one is easy to use

Results

Summary: Choosing a Dataset

- **Create your own dataset:**
  - Pros: Fully customized, fits your assignment exactly, shows you designed it yourself.
  - Cons: Takes time to generate manually (can use AI or Python to help).
- **Download online:**
  - Pros: Quick and ready to use.
  - Cons: Might need cleaning, less control, extra unused fields.

Recommendation:

- **Generate your own dataset (300–400 movies) using AI or Python.**
- It's easier to integrate with your recommender, explanations, and prompts, and it shows original work.

Prompt3

build recommendation logic – match user queries to movies using keywords or TF-IDF similarity.

Results

Recommendation Logic (TF-IDF)

Steps:

1. Combine movie fields (title, genre, cast, plot, year) into a single text per movie.
2. Use `TfidfVectorizer` to convert movie texts and the user query into vectors.
3. Compute cosine similarity between the query and each movie.
4. Sort movies by similarity and return the top results.

Python Code Snippet

python Copy code

```python
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import linear_kernel

# Combine movie fields into a single text
df['combined'] = df['title'] + ' ' + df['genre'] + ' ' + df['cast'] + ' ' + df['plot'] + ' ' + df[

# Fit TF-IDF vectorizer
vectorizer = TfidfVectorizer(stop_words='english')
tfidf_matrix = vectorizer.fit_transform(df['combined'])

# Function to recommend movies based on query
def recommend(query, top_k=5):
    q_vec = vectorizer.transform([query])
    cosine_sim = linear_kernel(q_vec, tfidf_matrix).flatten()
    top_idx = cosine_sim.argsort()[::-1][:top_k]
    return df.iloc[top_idx][['title', 'year', 'genre', 'cast', 'plot']]

# Example usage
recommend("light romantic comedies from the 1990s")
```

✅ This snippet gives the **top 5 most relevant movies** based on the user query.

Prompt4

Generate explanations – short, readable sentences explaining why each movie was recommended.

Here's how you can **generate short, readable explanations** for each recommended movie, matching your assignment requirements:

Results

Summary: Generating Explanations

- **Purpose:** Give a short, readable reason for each recommended movie.
- **Key points to include:**
  - Genre matches the user query
  - Cast matches (actor names in query)
  - Release year or decade matches
  - General theme similarity if no exact match
- **Implementation:**
  - Check query against movie fields (genre, cast, year)
  - Combine matches into a single sentence
  - Provide a default reason if nothing matches

Code snippet example:

python Copy code

```python
def explanation_for(query, movie):
    query_lower = query.lower()
    reasons = []
    for g in movie['genre'].lower().split(";"):
        if g in query_lower: reasons.append(f"matches genre '{g}'")
    if "1990" in query_lower and 1990 <= movie['year'] <= 1999:
        reasons.append("released in the 1990s")
    for actor in movie['cast'].split(","):
        if actor.strip().split()[0].lower() in query_lower:
            reasons.append(f"stars {actor.strip().split()[0]}")
    if not reasons: reasons.append("has similar themes to your query")
    return "Because it " + ", and it ".join(reasons) + "."
```

This produces **readable explanations** for each recommended movie.

Prompt5

Create a GUI – allow users to type queries and see recommendations.

Results

Here's a **summarized GUI explanation with code snippet:**

GUI for the Recommender System

Purpose:

- Let users type a query and get movie recommendations with explanations.

Components:

1. **Text box** – for user input.
2. **Button** – click to generate recommendations.
3. **Output area** – displays top movies and explanations.

Implementation: Use **Jupyter Notebook + ipywidgets** for a simple interactive interface.

Python Code Snippet

python Copy code

```python
import ipywidgets as widgets
from IPython.display import display, clear_output

# Input box and button
query_box = widgets.Text(
    value='light romantic comedies from the 1990s',
    placeholder='Type your movie request...',
    description='Query:',
    layout=widgets.Layout(width='80%')
)

button = widgets.Button(description='Recommend', button_style='success')
output = widgets.Output()
```

```python
# Function to display recommendations
def on_click(b):
    with output:
        clear_output()
        query = query_box.value
        recs = recommend(query)  # TF-IDF recommendation function
        recs['explanation'] = recs.apply(lambda r: explanation_for(query, r), axis=1)
        print(f"Top recommendations for: \"{query}\"")
        for i, r in recs.iterrows():
            print(f"\n🎬 {r.title} ({r.year}) — {r.genre}")
            print(f"Cast: {r.cast}")
            print(f"Plot: {r.plot}")
            print(f"Why: {r.explanation}")

# Link button to function
button.on_click(on_click)

# Display GUI
display(query_box, button, output)
```

✅ This creates an **interactive GUI** inside Jupyter Notebook or Colab, showing **top movies with explanations** when a query is entered and the button is clicked.

2. Explanations

Prompt1

- The structure of the movie dataset was designed with the help of this prompt in order to determine how the dataset should appear, I asked the AI to generate example columns like title, genre, cast, narrative, and year.
- A dataset of 400 synthetic movies was generated by re-creating the clear structure that the AI's response offered in Python.
- To enhance accuracy and diversity, I minimized some of the columns and ensured the dataset includes realistic names and a mix of genres.

Prompt2

- The goal of this prompt was to produce brief explanation templates for each movie's recommendation e.g., matching genre, decade, or actor.
- The templates were converted into the `explanation_for()` function in the code, allowing the system to show clear reasons for each recommendation.
- I changed and simplified certain AI-generated explanations to make them sound more natural because they were too long and repetitive.

Prompt3

- This prompt was used to ask the AI for pseudocode or guidelines for creating a movie recommender system that uses TF-IDF and similarity scoring.

- The AI result assisted me in developing the primary recommendation logic in Python, which involved TF-IDF vectorization and cosine similarity.

- I changed the algorithm to include a boost for specific decades such as the 1990s and to limit the results to the top five recommendations.

Prompt4

- This prompt was used in Google Colab to produce a GUI layout with ipywidgets, which included a text field and a button.
- The AI provided sample widget code, which I modified to make the interface simple and user-friendly for inputting queries.
- I changed button colors, labels, and placeholder text to make it clearer for users and to fit the Netflix-style theme.

Prompt5

- This prompt assisted in developing test queries to assess how well the recommender responded to various inputs like genres, years, and moods.
- The AI ideas were used as test cases for the recommender system, with screenshots captured for the report.
- To fit my generated dataset, I changed some of the AI's test queries with unique genres and fake names.

3. **Final reflections on what worked well or what could be improved in the prompt process.**
    - The use of AI prompts strongly simplified the design of the dataset, the creation of explanation templates, and the development of the recommendation logic. When TF-IDF and simple rule-based boosting were used together, the results were good and easy to understand for most user searches. The explanation templates were initially a little redundant, so I changed up the wording and supplied backup reasoning in cases where there was no exact match. I could make the system better in the future by restricting the length of explanations, developing additional prompts for matching cast members and plots, and carefully checking the prompts to ensure the output correctly fits the dataset.